Object Oriented Design and Implementation of a Military Aviation Spare Parts Inventory System

First Semester 2026-27
Project Oct/Nov 2026 | Document D2

SS G514 OBJECT ORIENTED ANALYSIS & DESIGN

Area: 40, Defence

STUDENT DETAILS	
Name	Milap Chaudhari
ID No.	2025H1030207P
Course	SS G514 Object Oriented Analysis & Design
Paper selected	The efficiency of using a tailored inventory management system in the military aviation industry

Abstract

This project implements a small console inventory system for an air force stores unit. The prototype models parts and serialized stock items, technician requisitions, approval by planners or commanders, stock reservation and issue, Aircraft on Ground (AOG) events, reorder proposals, and stock and AOG reports. It uses Java 17, CSV files, and a menu with four sample roles. The design separates the menu-facing InventoryService from domain objects and CSV repositories, and uses interchangeable fixed-level and minimum-maximum reorder policies. Six JUnit tests cover policy calculations, legal state changes, requisition lifecycle, and role restrictions. A deterministic 12-month synthetic demand comparison produced 11 stock-out events and a mean closed AOG duration of 442.3 hours for the fixed-level policy, compared with 7 events and 504.0 hours for minimum-maximum. This illustrative result shows a trade-off in the chosen sample; it does not establish operational performance. The work applies OOAD concepts to a constrained part of the inventory problem described in the selected aviation inventory study [1].

Contents

Section	Page
1. Introduction and background	3
2. Requirements and scope	3
3. System models	4
3.1 Use case model	4
3.2 Domain class model	4
3.3 Sequence model	5
3.4 Activity model	6
3.5 Stock item state model	6
4. Approach and architecture	7
5. Patterns and responsibility assignment	7
6. Design and implementation	8
7. Demonstration and results	8
8. Limitations and future work	9
9. Summary and conclusion	9
10. References	9

1. Introduction and background

Military aircraft readiness depends on having serviceable spare parts available when maintenance teams need them. A shortage can keep an aircraft on the ground, increase operating cost, and reduce squadron availability. The selected study describes a tailored inventory application for military aviation, implemented with Microsoft Access, and discusses its role in supply-chain decisions and aircraft availability [1].

This project applies object-oriented analysis and design to a limited implementation path through that problem. It converts the requirements from the earlier D1 analysis into a Java console application with stock transactions, requisitions, AOG events, reorder proposals, reports, and role checks. The prototype uses fictional data and does not connect to a real maintenance-management system or military database.

Problem and objectives

The system must record catalogue parts and stock status, reserve and issue available parts against approved requisitions, track the time an aircraft remains AOG, propose replenishment using replaceable policies, and provide stock and AOG duration reports. The implementation objective is a repeatable console demonstration of the missing-part path, supported by a small test suite and a comparison of two reorder strategies.

Actors

Actor	Primary responsibilities
Store keeper	Maintain the catalogue, receive stock, issue approved requisitions, record repair returns, and view reports.
Technician	Raise requisitions, declare AOG, and record when an issued part is fitted.
Planner	Approve or reject non-critical requisitions, review reorder proposals, select a reorder policy, and view reports.
Commander	Approve critical-part requisitions and view AOG and stock reports.

2. Requirements and scope

The implementation concentrates on the requirements needed for one complete inventory-to-maintenance path. The role check is a local demonstration mechanism; it maps a sample login name to a role and does not authenticate passwords.

ID	Requirement represented in the prototype
FR1	Create and update catalogue entries with number, description, unit cost, criticality, type, and reorder thresholds.
FR2	Receive stock; create individual serials for stock items and preserve serviceability states.
FR3	Reserve and issue only available stock against an approved requisition.
FR4	Raise a requisition with a priority, aircraft tail, and one or more part lines.
FR5	Approve or reject requisitions; commander approval is required for critical parts.
FR6	Open an AOG event for a grounded aircraft and close it when its allocated part is fitted.
FR7	Guard stock item state transitions, including repair return.
FR8	Create one open reorder proposal per part using fixed-level or minimum-maximum policy.
FR9	Display stock and AOG duration reports and save them as text files.

ID	Requirement represented in the prototype
FR10	Restrict actions by role and record inventory and workflow transactions in an audit file.

Scope and assumptions

The program uses one local store and made-up catalogue data. CSV files hold the catalogue, stock items, requisitions, proposals, AOG events, and audit entries. The console represents one operator at a time. Supplier purchasing, a graphical interface, real aircraft records, multi-warehouse management, and maintenance-system integration are outside the project scope, consistent with the D1 plan.

The implementation represents an aircraft through its tail number in requisitions and AOG events rather than a persisted Aircraft object. This keeps the prototype compact while preserving the identifier needed to link a fitted item to the matching AOG event.

3. System models

3.1 Use case model

The four actors use a single inventory system boundary. The role check in InventoryService rejects operations that are not assigned to the logged-in role. A requisition may be raised by a technician; approvals are handled by logistics or command; store actions are performed by the keeper.

Inventory management system use cases

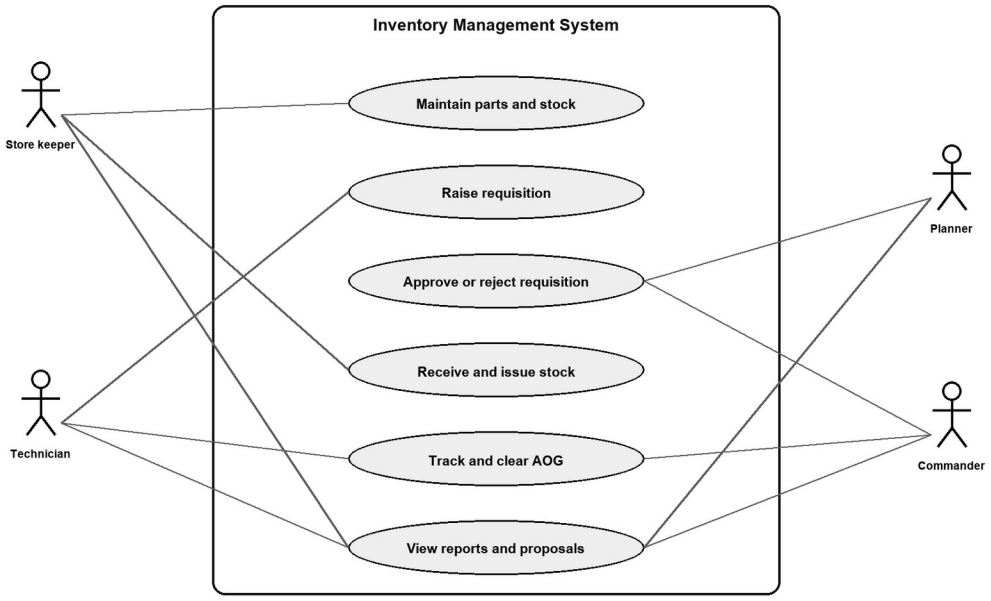


Figure 1. Use case model for the console prototype.

3.2 Domain class model

Part stores catalogue attributes; StockItem stores the serial and current status. Requisition contains one or more RequisitionLine values and records its priority, tail number, state, and allocated serials. AOGEvent stores a tail, cause, start and optional end time. InventoryService coordinates these objects and delegates persistence to InventoryRepository. The Aircraft shown as a domain concept in Figure 2 is represented in code by its tail number.

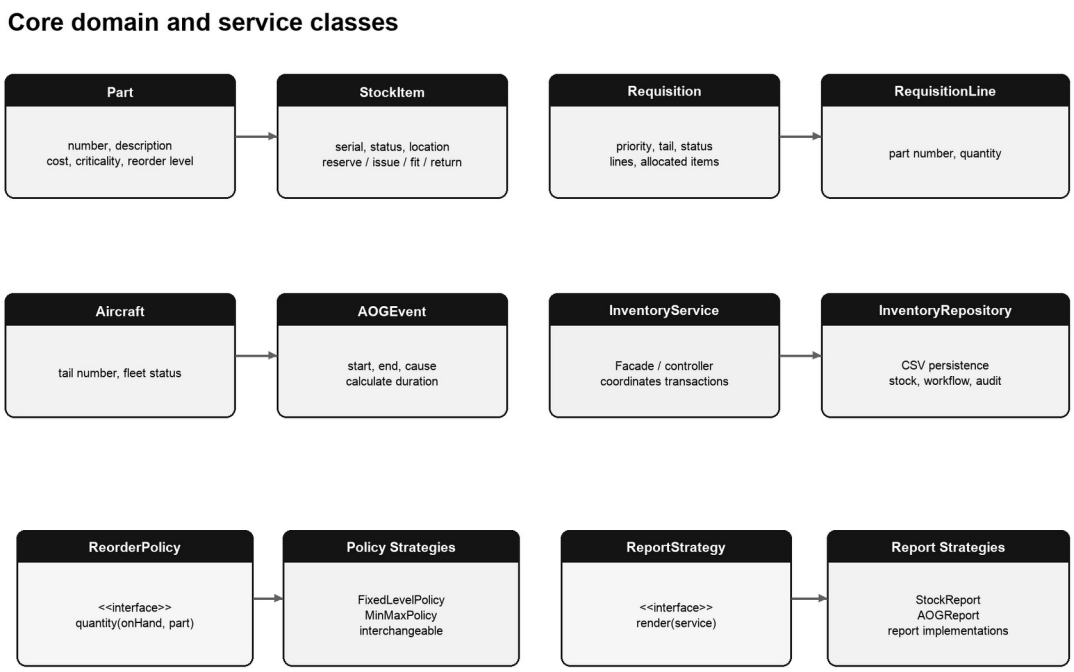


Figure 2. Main domain and service classes.

ReorderPolicy and ReportStrategy are interfaces. The fixed-level and minimum-maximum policies implement quantity calculations; StockReport and AOGReport produce the two report types. The repository keeps file access outside the domain transitions.

3.3 Sequence model

The sequence starts when a technician submits a grounded aircraft requisition for a missing critical part. InventoryService checks availability, records the AOG event and proposal, notifies the planner, and later processes the approval and receipt. Once the request is approved and issued, fitting the allocated serial closes the AOG event for the matching tail.

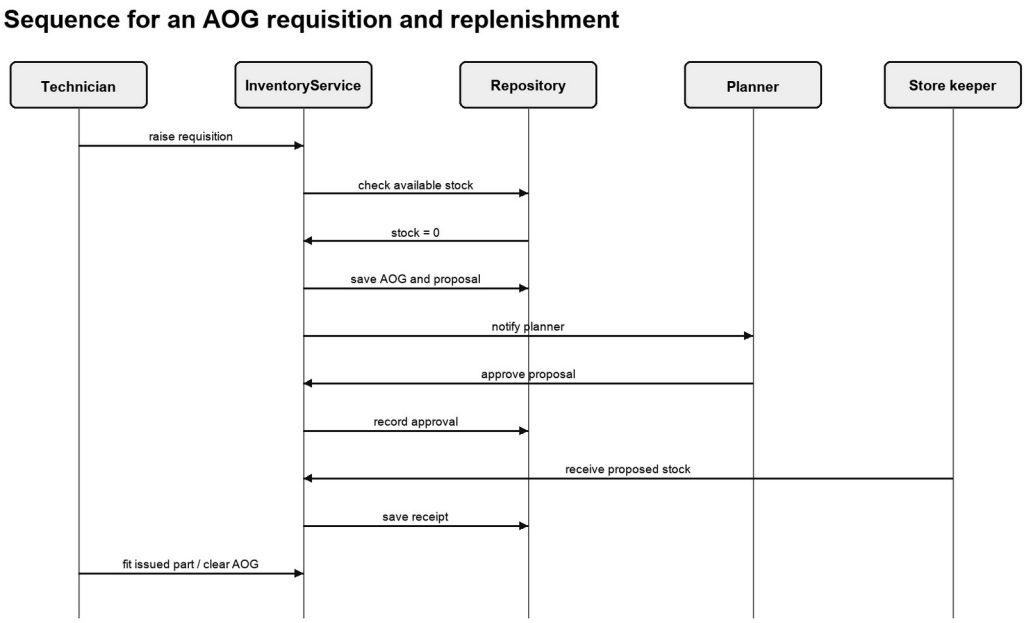


Figure 3. Sequence for an AOG requisition and replenishment.

3.4 Activity model

The activity model separates the available-stock path from the stock-out path. A non-critical request with enough stock is reserved and approved. A critical request waits for commander approval. A missing part can open an AOG event and create a reorder proposal; after the planner approves the proposal and the store receives the part, the technician fits it and closes the event.

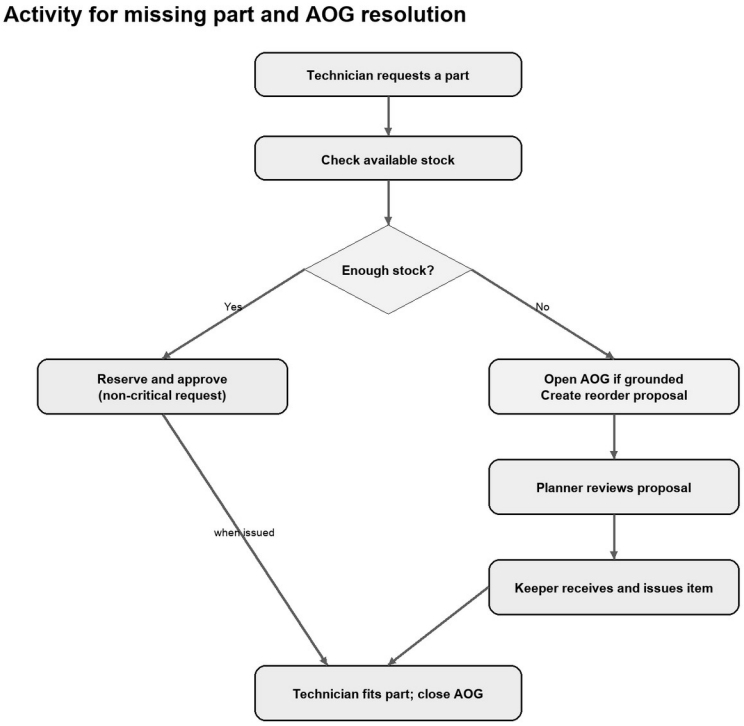


Figure 4. Activity for a missing part and AOG resolution.

3.5 Stock item state model

The StockItem transition method allows only defined moves. A reserved item can be released or issued; an issued item can be installed or marked unserviceable; an unserviceable item can return to stock after repair. Illegal transitions raise an error instead of silently changing inventory.

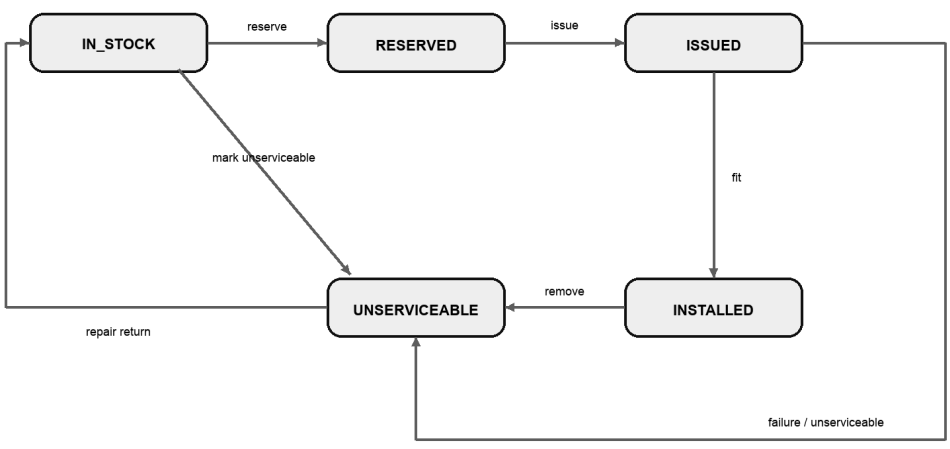


Figure 5. Stock item state transitions.

4. Approach and architecture

The work follows short iterations: first catalogue and stock handling, then requisitions and AOG, then policy and reports, and finally demonstration and tests. Each completed path uses fictional data so that the same steps can be repeated without external services.

Layer responsibilities

Layer	Responsibilities
Console menu	Reads user choices and prints status, errors, and reports.
InventoryService	Acts as the application controller and Facade for use cases.
Domain objects	Represent parts, serial items, requisitions, proposal states, and AOG duration.
Repository	Loads and writes CSV data and the audit log.
Policies and reports	Provide interchangeable reorder calculations and report generation.

Transaction rules

Stock is reserved before issue, and issue is accepted only from an approved requisition. Quantity checks are performed before reservation, and an AOG-priority request can release reservations held by a routine request; the routine request then returns to a waiting-for-stock state. Critical parts require commander approval. A proposal remains unique per part while it is open or approved.

5. Patterns and responsibility assignment

Patterns are used where they make a changing decision explicit or keep a dependency in one place. The implementation uses the GRASP Controller idea in InventoryService and assigns state and duration calculations to the objects that own those values [2].

Pattern	Location	Reason in this project
Facade / Controller	InventoryService	The console calls one service instead of coordinating repositories and domain objects itself.
Repository	InventoryRepository	CSV access is separated from business-state classes.
Strategy	ReorderPolicy and ReportStrategy	Policy calculations and report choices can be exchanged through interfaces.
State	StockItem and Requisition transitions	Allowed lifecycle changes are checked at the object that owns the state.
Observer	Service listeners	AOG and reorder events can notify interested listeners without embedding planner logic in stock objects.
Factory-style creation	Stock receiving	The receiver chooses a serial prefix from the part type at one creation point; a dedicated Factory Method can replace this as item subtypes grow.

A Singleton connection was not added. This version uses local CSV files and has no database connection or shared configuration service that needs a single global instance. Avoiding a global resource keeps the service easier to construct in tests.

6. Design and implementation

Catalogue and stock

The sample catalogue contains three fictional parts: a hydraulic filter, a fuel control unit, and a navigation relay. A StockItem represents one stock unit and uses a serial identifier; item status distinguishes in-stock, reserved, issued, installed, and unserviceable. Receiving and repair return update the item list, while issue changes reserved items to issued.

Requisitions and AOG

A request records one or more part lines, a routine, urgent, or AOG priority, and the aircraft tail. The service checks stock, reserves non-critical requests when possible, and leaves critical requests awaiting commander approval. AOG events store their opening and closure times and the serial that resolved them. AOG priority may preempt a routine reservation when supply is short.

Persistence and audit

File	Persisted records
parts.csv	Catalogue attributes and reorder thresholds.
stock.csv	Stock item serials and current states.
requisitions.csv	Lines, status, aircraft tail, priority, and allocations.
aog-events.csv	Cause, start/end timestamps, and resolving serial.
proposals.csv	Part, quantity, policy name, and proposal status.
audit.csv	Timestamp, user, transaction type, and transaction detail.

Policy definitions

The fixed-level policy orders max(0, reorder level minus available stock). The minimum-maximum policy orders enough to reach the maximum only when available stock is below the minimum. Both strategies are used by the same proposal workflow.

7. Demonstration and results

Console demonstration path

The included input file drives a repeatable missing-part scenario: set the policy, raise an AOG request for ROT-220 on AC-01, approve and receive the proposal, approve the critical requisition as commander, issue the part, fit its serial, view both reports, and compare the policies. The output file contains the captured console transcript.

Verification	Result
Java compilation	Compiled with a JDK 25 compiler using --release 17.
JUnit tests	6 tests passed; 0 failures and 0 errors in Maven Surefire.
End-to-end sample	AOG event and reorder proposal were created; receipt, approval, issue, fit, and report steps completed.
Policy comparison	Option 19 reports stock-out events and mean closed AOG duration for the two strategies.

Synthetic policy comparison

Policy	Stock-out events	Mean closed AOG duration
Fixed level	11	442.3 hours
Minimum-maximum	7	504.0 hours

On this synthetic demand sequence, minimum-maximum had fewer stock-out events but a longer mean duration among closed AOG events. The simulation uses a fixed 12-month demand sequence and a 30-day

replenishment lead time; it is a demonstration of comparison, not an estimate of real fleet performance. An AOG still open at the end of the simulation is excluded from the closed-event mean.

8. Limitations and future work

This prototype is deliberately limited to one local console and one store. It does not authenticate passwords, connect to suppliers, integrate with a maintenance-management system, or use real military data. Its synthetic demand comparison is intentionally small and does not model acquisition costs, variable lead times, cannibalisation, safety stock, or different aircraft types.

The CSV repository is suitable for a repeatable classroom demonstration but is not a multi-user transactional database. The console performs synchronous file writes and does not provide concurrent-user locking. The sample catalogue contains three parts rather than the larger dataset proposed in D1. The role mapping illustrates authorization boundaries but is not a security system.

Next extensions

- Move to SQLite transactions while preserving the Repository interface.
- Add a dedicated StockItemFactory and concrete Consumable and Rotable types if category-specific behavior grows.
- Store aircraft and maintenance events as explicit entities and link AOG resolution to requisition identifiers.
- Use a larger, clearly parameterized synthetic demand set and include unit cost and inventory holding cost in policy evaluation.
- Add authenticated users, concurrent transaction handling, and a graphical front end only if the project scope expands.

9. Summary and conclusion

The project turns the selected military aviation inventory problem into a runnable object-oriented console prototype. It covers catalogue maintenance, stock state changes, requisitions, role-based decisions, AOG timing, reorder proposals, reports, persistence, and a controlled comparison of two policies. The class and sequence models connect the analysis to the implementation; the tests cover core state, policy, and authorization rules.

The synthetic comparison favours minimum-maximum on stock-out count in the chosen workload, while the fixed-level policy records a shorter mean among the AOG events that close during the simulated year. These outcomes show why policy selection should be evaluated against both availability and downtime using representative demand. They do not establish a general operational advantage. The main course result is a modular design that can be extended beyond a single database file while remaining small enough to demonstrate.

10. References

[1] Al-Momani H., Al Meanazel O.T., Kwaldeh E., Alaween A., Khasaleh A., Qamar A.; The efficiency of using a tailored inventory management system in the military aviation industry; Heliyon; Volume 6: Number 7, 2020, Article e04424. DOI: 10.1016/j.heliyon.2020.e04424.

[2] Larman C.; Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development; Prentice Hall; 3rd Edition, 2004, 703 pp.

[3] Gamma E., Helm R., Johnson R., Vlissides J.; Design Patterns: Elements of Reusable Object-Oriented Software; Addison-Wesley; 1st Edition, 1994, 395 pp.

[4] Booch G., Maksimchuk R.A., Engle M.W., Young B.J., Conallen J., Houston K.A.; Object-Oriented Analysis and Design with Applications; Addison-Wesley; 3rd Edition, 2007, 691 pp.

[5] Fowler M.; Patterns of Enterprise Application Architecture; Addison-Wesley; 1st Edition, 2002, 533 pp.

Internet sources

[6] Selected paper: https://pmc.ncbi.nlm.nih.gov/articles/PMC7364037/

[7] DOI record: https://doi.org/10.1016/j.heliyon.2020.e04424